

Unit-2

Advance Prompt Engineering

By: Dr. A. R. Welekar

Advanced Prompt Engineering for Engineers

- As engineers, you're used to designing systems, tuning parameters, and optimizing behavior.
- Prompt engineering is very similar—you're designing how a model behaves by carefully choosing inputs (prompts) and configuring parameters.

1. Chain-of-Thought Prompting (Step-by-Step Thinking)

It's like writing the steps in solving an engineering problem before giving the final answer. This makes the model simulate logical thinking, improving accuracy on complex tasks.

Analogy:

When you solve a circuit problem, you:

- Draw the diagram
- Apply KVL/KCL
- Solve equations
- Then find current/voltage
- Chain-of-thought prompting makes the AI do the same.

1. Chain-of-Thought Prompting (Step-by-Step Thinking)

Example Prompt:

Q: A train travels 60 km at 60 km/h, then 120 km at 120 km/h. What is the average speed?

A: Let's think step by step.

Time1 = $60 / 60 = 1$ hr

Time2 = $120 / 120 = 1$ hr

Total Distance = $60 + 120 = 180$ km

Total Time = $1 + 1 = 2$ hrs

Average Speed = $180 / 2 = 90$ km/h

1. Chain-of-Thought Prompting (Step-by-Step Thinking)

Engineering Use Case:

- Debugging code
- Multistage calculation (stress analysis, network routing)
- Step-by-step algorithm breakdown

2. Temperature, Top-p, and Max Tokens

These are generation hyperparameters, like knobs on a control system.

Parameter	Controls	Analogy in Engineering	Example Effect
Temperature	Randomness	Noise in control signal	0.1 = deterministic, 1.0 = creative
Top-p	Probabilistic sampling	Trimming low-signal frequencies	0.9 = only top 90% probable words
Max tokens	Output length	Memory buffer limit	50 = cut after 50 words

2. Temperature, Top-p, and Max Tokens

Example:

```
{  
  "prompt": "Explain PID control to a 2nd-year engineering student",  
  "temperature": 0.5,  
  "top_p": 0.8,  
  "max_tokens": 150  
}
```

Result:

- ✓ Output is informative, not too creative
- ✓ Short enough to fit UI
- ✓ Avoids generating weird outputs

Test it:

- ✓ Change temperature to 1.5 → Observe more poetic or unpredictable explanation. Good for brainstorming, bad for accuracy.

3. Prompt Tuning & Embedding-Based Prompts

Prompt Tuning:

- Prompt Tuning is a parameter-efficient fine-tuning method for adapting large pre-trained language models (LLMs) like GPT, T5, or BERT to downstream tasks without modifying the model weights.
- Instead of updating millions (or billions) of model parameters, only a small number of prompt embeddings (called soft prompts) are learned and optimized for the specific task.
- Instead of changing the entire model's weights (as in traditional fine-tuning), you learn a trainable prompt—essentially a vector that is prepended to the model's input.
- The model learns how to best respond when this specific prompt vector is used.

3. Prompt Tuning & Embedding-Based Prompts

Why Prompt Tuning?

➤ Traditional fine-tuning:

- ✓ Requires updating the entire model for each new task.
- ✓ Consumes lots of compute, memory, and time.
- ✓ Needs to store multiple full-sized models for multiple tasks.

➤ Prompt tuning solves this by:

- ✓ Training only a few thousand parameters (soft prompts).
- ✓ Keeping the original model frozen.
- ✓ xEnabling task-specific adaptation with minimal resource usage.

3. Prompt Tuning & Embedding-Based Prompts

1. The prompt is not natural language like "Summarize this paragraph", but a set of trainable embeddings—vectors in the model's input space.
2. These embeddings are inserted into the model's input before the actual task input.
3. During training:
 - Only the prompt embeddings are updated.
 - The language model remains unchanged.

Advantages:

- Very efficient – Much fewer parameters to train.
- Model-agnostic – Works with any pre-trained transformer.
- Useful for multiple tasks – You can learn separate prompts for different domains.

3. Prompt Tuning & Embedding-Based Prompts

How Does Prompt Tuning Work?

◆ Step 1: Pre-trained Language Model

You begin with a frozen pre-trained model like GPT-2 or T5. It already knows how to understand and generate language.

◆ Step 2: Add Soft Prompts

- ✓ Introduce a set of learnable embeddings (called soft prompts).
- ✓ These are vectors (not words!) inserted before the actual input.
- ✓ Example:

[P1][P2][P3] "Translate this sentence to French: Hello"

3. Prompt Tuning & Embedding-Based Prompts

1. Embedding-based prompts are input prompts represented as dense vectors (embeddings) instead of plain text tokens. They are also called soft prompts.
2. Traditional prompts: "Translate English to French: Hello"
3. Embedding-based prompts: $[E_1, E_2, E_3, \dots, E_k]$ where each E is a trainable embedding vector.
4. These embedding vectors are prepended to the model's input, and they condition the model's behavior.

3. Prompt Tuning & Embedding-Based Prompts

1. Create k soft prompt vectors $(E_1, E_2, \dots, E_k)$ – learnable parameters.
2. Concatenate them with token embeddings of the actual input sequence.
3. Input this combined vector sequence into the frozen LLM.
4. Let:
 - $x = [x_1, x_2, \dots, x_n]$ be tokenized input.
 - $P = [p_1, p_2, \dots, p_k]$ be learned prompt embeddings.

Then the model input becomes:

$[p_1, p_2, \dots, p_k, e(x_1), e(x_2), \dots, e(x_n)]$

Only the prompt P is updated during training.

3. Prompt Tuning & Embedding-Based Prompts

1. Create k soft prompt vectors $(E_1, E_2, \dots, E_k)$ – learnable parameters.
2. Concatenate them with token embeddings of the actual input sequence.
3. Input this combined vector sequence into the frozen LLM.
4. Let:
 - $x = [x_1, x_2, \dots, x_n]$ be tokenized input.
 - $P = [p_1, p_2, \dots, p_k]$ be learned prompt embeddings.

Then the model input becomes:

$[p_1, p_2, \dots, p_k, e(x_1), e(x_2), \dots, e(x_n)]$

Only the prompt P is updated during training.

4. Persona Creation through Prompt Control

- Persona creation through prompt control refers to the process of shaping how an AI model (like ChatGPT) behaves, speaks, or responds by using carefully crafted prompts.
- It is a powerful technique used to simulate distinct "personalities" or "roles" in an LLM (Large Language Model), even without changing the model's internal parameters.

4. Persona Creation through Prompt Control

A persona in the context of LLMs is a synthetic character with:

- A specific tone (friendly, sarcastic, formal)
- Defined knowledge scope (e.g., a doctor, historian, fitness coach)
- A unique style of communication
- Preset beliefs, goals, or limitations

Example:

"You are a helpful, patient math tutor who always uses real-world analogies to explain complex concepts."

4. Persona Creation through Prompt Control

❖ Prompt Control: The Engine Behind Persona Creation

- Prompt control is the method of engineering prompts to control how the AI:
- Interprets the situation
- Speaks/responds
- Handles knowledge and interaction

There are three major techniques for persona creation through prompt control:

1. Instruction-based Prompting
2. Few-shot Prompting with Examples
3. Soft Prompting / Prompt Tuning (Advanced)

4. Persona Creation through Prompt Control

❖ 1. Instruction-based Prompting

You provide explicit instructions in the prompt to guide behavior.

Example:

"Act as a witty, sarcastic movie critic who hates cliches. Review the movie Titanic."

The model will emulate that persona just from the instruction. This is zero-shot persona control, needing no training.

4. Persona Creation through Prompt Control

2. Few-shot Prompting with Examples

Provide examples in the prompt to demonstrate how the persona behaves. This helps reinforce the style or tone.

Prompt:

You are 'ChefAI', a humorous celebrity chef bot.

User: What's the secret to perfect pasta?

ChefAI: Ah, darling, it's not the sauce — it's the drama! Salt that water like your ex just texted you.

User: How do I boil an egg?

ChefAI:

→ The model continues with that tone automatically.

4. Persona Creation through Prompt Control

3. Soft Prompting / Prompt Tuning (Advanced)

Instead of written prompts, you train embedding vectors (soft prompts) that "nudge" the model into a persona using learned latent instructions.

- Requires access to model internals and training loop.
- Uses a fixed model; only prompt embeddings are updated.
- More efficient and parameter-light than full fine-tuning.

Use case: You train a 20-token soft prompt that makes the model act like a Shakespearean poet without ever mentioning "Shakespeare" in the input.

4. Persona Creation through Prompt Control

Practical Examples

◆ Customer Support Bot

"You are an empathetic customer care agent. Always apologize first, then offer solutions."

◆ AI Research Tutor

"You are a PhD-level AI professor. Answer in detailed, technical language with equations if necessary."

◆ Language Learning Partner

"You are a Spanish-speaking friend. Always reply in Spanish with simple vocabulary suitable for beginners."

5. Prompt Design Patterns

- Prompt Design Patterns are repeatable, structured techniques used in crafting effective prompts for large language models (LLMs) like ChatGPT, GPT-4, Claude, etc.
- These patterns help maximize the performance of the model by guiding it toward producing accurate, relevant, and creative outputs. Just like software design patterns solve recurring design problems in code, prompt design patterns solve recurring interaction problems between users and LLMs.

Why PDP?

- Increase reliability and consistency of outputs
- Reduce hallucination and ambiguity
- Enable complex tasks through composition
- Enhance zero-shot, one-shot, or few-shot learning performance
- Simplify reuse of effective prompt structures

5. Prompt Design Patterns

◆ Categories of Prompt Design Patterns:

1. Instructional Patterns

These guide the LLM by giving it clear, directive instructions.

a. Direct Instruction Pattern

Translate this sentence to French: "I am learning AI."

b. Step-by-Step Pattern (Chain-of-Thought Prompting)

Solve this math problem step by step: What is 19×23 ?

c. Do-Then-Reflect Pattern

Write a short story about a robot that wants to be human. Then reflect on the moral of the story.

5. Prompt Design Patterns

◆ Categories of Prompt Design Patterns:

2. Few-shot Learning Patterns

Demonstrate the task with a few examples to guide model behavior.

a. Few-shot Classification Pattern

Input: The food was terrible and cold.

Sentiment: Negative

Input: The view was stunning and service excellent.

Sentiment: Positive

Input: It took too long to arrive.

Sentiment:

5. Prompt Design Patterns

◆ Categories of Prompt Design Patterns:

2. Few-shot Learning Patterns

Demonstrate the task with a few examples to guide model behavior.

b. Few-shot Translation Pattern

English: Hello

Spanish: Hola

English: Thank you

Spanish: Gracias

English: How are you?

Spanish:

5. Prompt Design Patterns

◆ Categories of Prompt Design Patterns:

3. Role-Playing Patterns

Assign a persona or role to the LLM to shape its tone, style, or behavior.

a. Act-as Pattern

You are an experienced career coach. Advise a recent graduate on how to prepare for interviews.

b. Persona Injection Pattern

You are Albert Einstein, explaining the theory of relativity to a high school student.

5. Prompt Design Patterns

◆ Categories of Prompt Design Patterns:

4. Format Constraints Patterns

Control the structure of the output.

a. Structured Output Pattern

List the pros and cons of electric vehicles in a table format with two columns.

b. JSON Output Pattern

Give me details about the Eiffel Tower in JSON format with keys: name, location, height_meters, year_built.

5. Prompt Design Patterns

◆ Categories of Prompt Design Patterns:

5. Reflective & Self-Improving Patterns

Help the model reason or critique its own output.

a. Self-Critique Pattern

Write a paragraph about climate change. Then critique your own paragraph for balance and accuracy.

b. Double Check Pattern

Answer the question: What is the capital of Australia? Then double-check your answer and explain

5. Prompt Design Patterns

◆ Categories of Prompt Design Patterns:

6. Decomposition Patterns

Break down complex tasks into substeps.

a. Plan-and-Execute Pattern

Plan how to write a research paper on AI in education. Then write the introduction based on your plan.

b. Divide-and-Conquer Pattern

Divide the topic “History of the Internet” into 3 parts. Then explain each part in 2 sentences.

5. Prompt Design Patterns

◆ Categories of Prompt Design Patterns:

7. Multi-turn Patterns

Used in dialogue-based prompting for conversation or task refinement.

a. Clarify-Then-Answer Pattern

Before answering, ask me any clarifying questions you need about my request to write a speech.

b. Memory Recall Pattern

Earlier, I said I was organizing a hackathon. Now give me a checklist based on that.

5. Prompt Design Patterns

◆ Combining Patterns:

You can often chain or combine multiple prompt patterns to increase reliability:

(Act-as Pattern) You are a software architect.

(Decomposition Pattern) Break down the steps to build a REST API in Python.

(Format Constraint Pattern) Output each step as a numbered list.

5. Prompt Design Patterns

◆ Example: Resume Tailoring

You are a professional resume writer.

Given this job description and resume, suggest 5 improvements to align them.

Job Description: Software Engineer with experience in Python, REST APIs, and AWS.

Resume:

<Name>: John Doe

<Skills>: Java, MySQL, basic Python

<Experience>: Interned at a bank as backend developer